

Tener tiempo es el mejor activo que puedes tener para un TFG. Muchos alumnos lo hacen con prisas y eso se nota en tres puntos: el **Estado del Arte** es pobre, el **Diseño** es inexistente (saltan directo a programar) y las **Pruebas** son anecdóticas.

Dado que quieres un trabajo de **Matrícula de Honor (o Sobresaliente alto)**, esta es la estructura canónica de una **Memoria de Ingeniería Informática**. No es solo un "trabajo", es un proyecto de ingeniería completo.

Aquí tienes el esquema maestro. Puedes copiarlo y pegarlo en tu documento para tener el esqueleto.

ÍNDICE PROPUESTO PARA UN TFG DE INGENIERÍA INFORMÁTICA

A. PAGINACIÓN PRELIMINAR (Números romanos: i, ii, iii...)

- **Resumen (Abstract):** En español e inglés. Vital. Es lo único que mucha gente leerá. Se escribe al final.
- **Palabras Clave (Keywords):** 3-5 términos (ej: *Machine Learning, Python, API REST, Ciberseguridad*).
- **Índice General:** La estructura completa.
- **Índice de Figuras:** Lista de todas las imágenes/gráficos.
- **Índice de Tablas:** Lista de todas las tablas.
- *(Opcional) Glosario de términos y acrónimos.*

1. INTRODUCCIÓN

El objetivo aquí es vender el proyecto. ¿Por qué existe?

- **1.1. Contexto y Justificación:** ¿De dónde nace la idea? ¿Qué problema real resuelve?
- **1.2. Motivación:** ¿Por qué es relevante para la ingeniería informática?
- **1.3. Objetivos: (CRÍTICO)**
 - *Objetivo General:* Una frase que resume la meta.
 - *Objetivos Específicos:* Lista de hitos medibles (ej: "Diseñar una BD...", "Implementar un algoritmo de...", "Validar el sistema mediante...").

- **1.4. Estructura del documento:** Breve párrafo explicando qué hay en cada capítulo.

2. ESTADO DEL ARTE (O MARCO TEÓRICO)

Demuestra que has leído y sabes lo que existe en el mercado. Aquí es donde se nota si tienes tiempo.

- **2.1. Conceptos Teóricos:** Explicación breve de las tecnologías base (¿Qué es Blockchain? ¿Qué es una red neuronal convolucional?). No copies de Wikipedia, sintetiza.
- **2.2. Soluciones existentes:** ¿Qué otras apps/sistemas hacen lo mismo?
- **2.3. Análisis comparativo:** Tabla comparando tu propuesta con las existentes. ¿En qué mejora tu TFG a lo que ya hay?

3. GESTIÓN DEL PROYECTO (Planificación)

Demuestra que eres un ingeniero organizado, no solo un programador.

- **3.1. Metodología de trabajo:** ¿Usarás Scrum? ¿Kanban? ¿Cascada? Explica por qué.
- **3.2. Planificación temporal:** Diagrama de Gantt. (Tareas vs. Tiempo).
- **3.3. Análisis de Riesgos:** ¿Qué pasa si falla la API externa? ¿Si pierdes datos? Matriz de riesgos y planes de contingencia.
- **3.4. Presupuesto:** Aunque no cobres, calcula cuánto costaría el proyecto en el mundo real (Tus horas x Precio hora ingeniero + Licencias + Hardware).

4. ANÁLISIS DEL SISTEMA

El "Qué" vamos a hacer.

- **4.1. Requisitos Funcionales:** Lista detallada de lo que el sistema DEBE hacer (ej: "El sistema debe permitir login con Google").
- **4.2. Requisitos No Funcionales:** Rendimiento, seguridad, escalabilidad, usabilidad.
- **4.3. Casos de Uso:** Diagramas de Casos de Uso (UML) y descripción de actores.

5. DISEÑO DEL SISTEMA

El "Cómo" lo vamos a hacer. Este es el capítulo más técnico antes del código.

- **5.1. Arquitectura del Sistema:** Diagrama de arquitectura (Cliente-Servidor, Microservicios, MVC).
- **5.2. Diseño de Datos:** Modelo Entidad-Relación (E/R) y esquema de Base de Datos.
- **5.3. Diseño de Clases/Componentes:** Diagramas de clases UML principales.
- **5.4. Diseño de Interfaz (UI/UX):** Mockups o prototipos de las pantallas principales.

6. IMPLEMENTACIÓN Y DESARROLLO

No pegues todo el código. Explica los desafíos.

- **6.1. Entorno de desarrollo:** IDEs, versiones de lenguajes, frameworks usados.
- **6.2. Estructura del proyecto:** Árbol de directorios explicado.
- **6.3. Detalles de implementación:** Explica los algoritmos complejos o las decisiones difíciles. "Se eligió la librería X sobre la Y porque...".
- **6.4. Control de versiones:** Breve mención al uso de Git/GitHub (ramas, commits).

7. PRUEBAS Y VALIDACIÓN

Aquí diferencias un trabajo amateur de uno profesional.

- **7.1. Estrategia de pruebas:** ¿Qué vas a probar?
- **7.2. Pruebas Unitarias:** Tests de código (JUnit, PyTest, etc.).
- **7.3. Pruebas de Integración/Funcionales:** ¿Funciona todo junto?
- **7.4. Pruebas de Rendimiento/Carga:** (Si aplica).
- **7.5. Validación de usuarios:** (Opcional) Resultados de encuestas o pruebas con usuarios reales.

8. CONCLUSIONES Y TRABAJO FUTURO

El cierre.

- **8.1. Conclusiones:** Repasa los objetivos del capítulo 1 y di si se cumplieron. (La "Regla del Espejo").
- **8.2. Lecciones aprendidas:** ¿Qué has aprendido técnicamente y personalmente?

- **8.3. Líneas de trabajo futuro:** ¿Qué le falta al proyecto? ¿Qué se podría añadir en una versión 2.0?

9. BIBLIOGRAFÍA

- Lista generada (con Zotero/Mendeley) en formato IEEE.

10. ANEXOS (Apéndices)

- **A. Manual de Instalación/Despliegue:** Cómo ponerlo en marcha.
- **B. Manual de Usuario:** Guía rápida de uso.
- **(Opcional) Fragmentos de código clave:** Solo si es muy necesario.

Consejos para aprovechar tu tiempo extra:

1. **La "Regla del Espejo":** El tribunal mirará primero tus **Objetivos** (Capítulo 1) y luego tus **Conclusiones** (Capítulo 8). Tienen que coincidir perfectamente. Si prometiste "Optimizar la base de datos" en los objetivos, en las conclusiones debe poner "Se optimizó la base de datos logrando una reducción del 20% en tiempos de consulta".
2. **Dale cariño al capítulo 4 y 5:** Muchos alumnos hacen un código genial pero una documentación pobre. Un buen diagrama de arquitectura o un buen modelo de datos (E/R) bien explicado vale oro para la nota.
3. **El Presupuesto:** Aunque parezca burocracia, a los ingenieros nos evalúan por nuestra capacidad de dimensionar costes. No lo hagas en 5 minutos.

¿Tienes ya una idea clara del **tema** de tu TFG o estás buscando inspiración todavía? Dependiendo del tema (IA, Web, App Móvil, Ciberseguridad), este índice se puede adaptar ligeramente.